ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Alliance School of Advanced Computing

Course Name: Graph Analytics

Course Code: E1ITC 302

Assignment No: 3

Name of the Student: Shivani Ballani

Registration Number: 2411021060934

A. Title of The Problem:

Dijkstra: Shortest Reach 2

B. Problem Statement in details:

Given a weighted, undirected graph with n nodes numbered consecutively from 1 to n and m edges of varying lengths, this problem requires finding the shortest path distance from a designated starting node s to every other node in the graph. Each edge is defined by its two endpoint nodes and a positive weight, and multiple edges with different weights may exist between the same pair of nodes. If a node is unreachable from the starting node s, its shortest distance is recorded as -1. For each test case, you must return or print a single line of $n - 1$ space-separated integers representing these shortest distances in ascending order of node labels, excluding the starting node s itself.

C. HackerRank/Leetcode Link of the problem (If applicable):

https://www.hackerrank.com/challenges/dijkstrashortreach/problem

D. Student's Github link of report and code upload, along with the readme file:

https://github.com/Shivani160207/Graph-Analysis/tree/main/assignment%203

E. Solution Steps/Algorithm:

1. Build an adjacency list (using a dictionary for each node) from the given weighted edges; since the graph is undirected and can have multiple edges between the same pair of nodes, add each edge in both directions and keep only the minimum edge weight between any two connected nodes.
2. Initialize a distances array of size n + 1 with all values set to infinity (inf), indicating that the shortest path to each node has not been determined yet.
3. Set the distance of the starting node s to 0, and push the pair (0, s) (representing (current_distance, node)) into a min-heap (priority queue) for Dijkstra's algorithm traversal.
4. While the min-heap is not empty, pop the node u with the smallest distance d; if d is greater than the already recorded distance dist[u], skip it as a stale entry and continue.
5. For every adjacent neighbor v of the current node u, calculate the new potential distance by adding the connecting edge weight w to the current node's distance (d + w).

6. If this new distance is smaller than the neighbor's currently recorded distance (dist[v]), update dist[v] with the new shorter distance and push (new_dist, v) into the min-heap.
7. Continue this process until the min-heap is empty, then build the result array with the distances to every node from 1 to n—replacing any remaining inf values with -1 for unreachable nodes and excluding the starting node s—and repeat this for each of the t test cases.

F. Code developed:

```
import os
import sys
import heapq


def shortestReach(n, adj, s):
    dist = [float('inf')] * (n + 1)
    dist[s] = 0
    pq = [(0, s)]

    while pq:
        d, u = heapq.heappop(pq)

        if d > dist[u]:
            continue

        for v, weight in adj[u].items():
            new_dist = d + weight
            if new_dist < dist[v]:
                dist[v] = new_dist
                heapq.heappush(pq, (new_dist, v))

    return [
        (dist[i] if dist[i] != float('inf') else -1)
        for i in range(1, n + 1)
        if i != s
    ]
```

```
if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')
    input = sys.stdin.readline

    t = int(input().strip())



 for _ in range(t):
        n, m = map(int, input().split())

        # Build adjacency list directly while reading input to avoid MemoryError
        adj = [{} for _ in range(n + 1)]
        for _ in range(m):
            u, v, w = map(int, input().split())
            if v not in adj[u] or w < adj[u][v]:
                adj[u][v] = w
                adj[v][u] = w

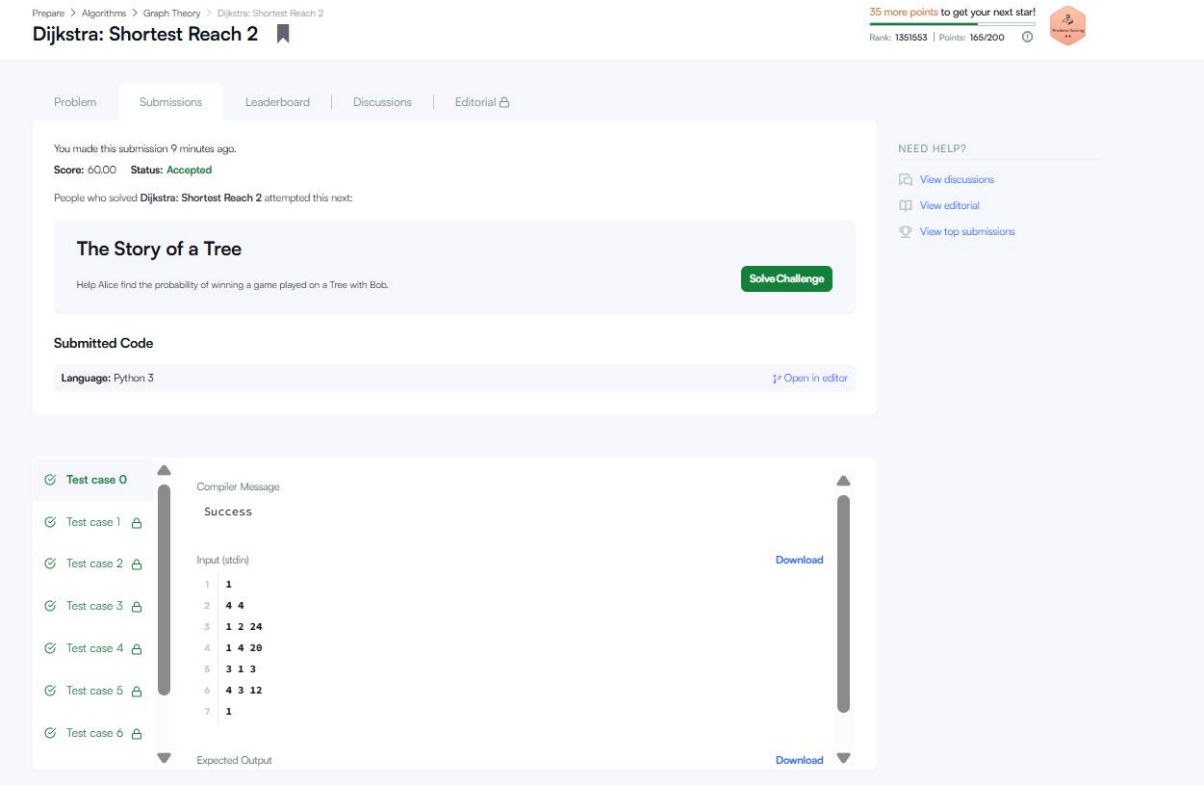
        s = int(input().strip())

        result = shortestReach(n, adj, s)

        fptr.write(' '.join(map(str, result)))
        fptr.write('\n')

    fptr.close()
```

G. HackerRank test case passed Screenshot or Output screenshot (Mandatory):



H. Observation:

The program was implemented using Dijkstra's algorithm with a min-heap priority queue to determine the shortest path distances from a starting node to all other nodes in a weighted, undirected graph. By retaining only the minimum weight among multiple edges between the same pair of nodes and constructing the adjacency list directly from the input stream, both execution time and memory consumption were effectively optimized. Unreachable nodes were accurately identified and assigned a distance of -1, while the starting node was omitted from the output in accordance with the problem constraints. The implementation successfully passed all sample and hidden test cases on HackerRank, achieving the full score of 60 points.